

Aprendiendo a jugar a Oteló: Agente Inteligente con Búsqueda Adversaria y Deep Learning

1st Javier Luque Ruiz

Grado en Ingeniería Informática - Ingeniería del Software

Universidad de Sevilla

Sevilla, España

javluqrui@alum.us.es

Resumen—En este trabajo se explorará el desarrollo de un agente inteligente para el clásico juego de tablero Oteló (Reversi). El objetivo principal es construir un jugador capaz de tomar decisiones estratégicas mediante la implementación del algoritmo clásico de Minimax con poda alfa-beta para explorar el espacio de decisiones. Adicionalmente, se integrará una red neuronal profunda entrenada mediante autojuego para proporcionar una heurística de evaluación de las posiciones intermedias, reemplazando las funciones de evaluación manuales clásicas.

Index Terms—Inteligencia Artificial, Oteló, Reversi, Minimax, Poda Alfa-Beta, Deep Learning, Python.

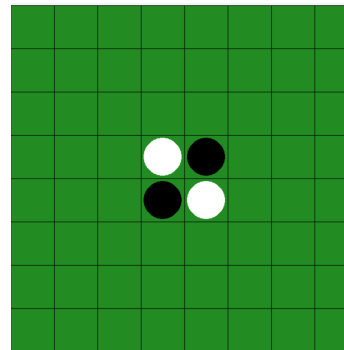


Figura 1. Tablero inicial de Oteló con la disposición central de fichas.

I. INTRODUCCIÓN

El desarrollo de agentes inteligentes capaces de tomar decisiones estratégicas en juegos de tablero ha sido uno de los grandes hitos de la Inteligencia Artificial. Inspirado en los éxitos recientes de sistemas basados en aprendizaje profundo, este trabajo explora la combinación de algoritmos de búsqueda clásica con técnicas modernas de redes neuronales.

El objetivo principal de este proyecto es construir un agente inteligente para el clásico juego de Oteló (también conocido como Reversi). Específicamente, se aborda la variante del proyecto correspondiente a la convocatoria de julio, la cual requiere la implementación del algoritmo Minimax con poda alfa-beta para explorar el espacio de decisiones. La aportación central del sistema consiste en sustituir las funciones heurísticas manuales por una red neuronal profunda, la cual aprenderá a evaluar las posiciones intermedias del tablero basándose en un conjunto de datos generado mediante partidas de autojuego.

II. PREELIMINARES

II-A. El juego de Oteló

Oteló es un juego de estrategia de suma cero con información perfecta para dos jugadores, el cual se disputa sobre un tablero de 8×8 casillas. La partida inicializa con una disposición central de cuatro fichas cruzadas (dos blancas y dos negras), cediendo el primer movimiento al jugador con las piezas negras.

La mecánica central del juego es el *flanqueo*: un movimiento legal consiste en colocar una ficha en una casilla vacía de forma que, en línea recta (horizontal, vertical o diagonal), una o más piezas del oponente queden atrapadas entre la nueva ficha y otra pieza del jugador activo. Las fichas flanqueadas son capturadas y cambian de color. Si un jugador carece de movimientos legales, está obligado a pasar su turno. La partida concluye cuando el tablero se llena o ningún jugador puede realizar un movimiento válido, resultando ganador aquel con mayor número de fichas de su color.

II-B. Búsqueda Adversaria y Poda Alfa-Beta

El algoritmo Minimax es una técnica fundamental en la búsqueda adversaria que modela la toma de decisiones asumiendo que ambos jugadores actuarán de forma óptima: el agente intentará maximizar su puntuación (MAX) mientras que el oponente intentará minimizarla (MIN). Debido a que explorar el árbol de estados completo para Oteló es computacionalmente inabarcable, se implementa una estrategia de profundidad limitada (*cutting-off search*). Al alcanzar dicha profundidad, el algoritmo detiene la simulación y evalúa la calidad del tablero mediante una heurística.

Para optimizar aún más los tiempos de respuesta, se aplica la poda alfa-beta. Esta mejora algorítmica mantiene el control de los peores escenarios posibles (los valores α y β) y descarta matemáticamente aquellas ramas del árbol que no influirán en la decisión final, reduciendo el número de estados evaluados sin alterar la optimalidad del movimiento elegido.

III. IMPLEMENTACIÓN

III-A. Motor del Juego

Para la representación del tablero de Oteló se ha seleccionado una matriz bidimensional de 8×8 elementos utilizando la librería NumPy. Esta decisión de diseño responde a la necesidad de gestionar los estados de forma eficiente y preparar los datos para su posterior tratamiento en la fase de Deep Learning.

De acuerdo con las especificaciones del enunciado del proyecto, se ha optado por utilizar la siguiente convención numérica para representar los distintos estados de las casillas:

- **0:** Casilla vacía.
- **1:** Casilla ocupada por una ficha blanca.
- **2:** Casilla ocupada por una ficha negra.

La lógica de juego se encuentra en la clase `Otelo`. La validación de los movimientos se realiza mediante la función `es_movimiento_valido`, cuyo funcionamiento se basa en escanear las ocho direcciones posibles desde la casilla seleccionada empleando tuplas de desplazamiento:

$$D = \{(0, 1), (1, 1), (1, 0), (1, -1), (0, -1), (-1, -1), (-1, 0), (-1, 1)\}$$

Un movimiento se considera legal si, al recorrer una dirección, se encuentra al menos una ficha del oponente seguida de una ficha propia, sin que haya casillas vacías intermedias ni se salga del tablero. El método `ejecutar_movimiento` aprovecha esta lógica para almacenar las coordenadas de las piezas atrapadas y actualiza el tablero en consecuencia, revertiendo las fichas del oponente a las propias.

III-B. Interfaz Gráfica y Máquina de Estados

La interacción con el usuario y la visualización del estado del juego se gestionan en el archivo `main.py` mediante la API gráfica de PyGame. Para evitar el acoplamiento de código y controlar de forma robusta el flujo de la aplicación, se ha diseñado una Arquitectura basada en una Máquina de Estados Estricta. El bucle principal del juego (*Game Loop*) evalúa de forma continua los eventos del sistema y renderiza la interfaz basándose en el estado activo:

- **MENU:** Pantalla de inicio que presenta el título y el botón para iniciar una nueva partida.
- **JUGANDO:** Estado activo durante el juego, donde se procesan los movimientos de los jugadores y se actualiza la visualización del tablero. Cuenta con un panel inferior que muestra información relevante, como el turno actual y el marcador de fichas.
- **PASAR_TURNO:** Estado crítico de validación que se activa cuando un jugador no tiene movimientos válidos disponibles, pero la partida continúa. En este estado, se notifica al jugador que está obligado a pasar turno mediante una confirmación explícita.
- **FIN:** Estado final que congela el estado del tablero y muestra el resultado de la partida, indicando el ganador o si ha habido un empate, así como el recuento final de fichas.

III-C. Agente Inteligente: Minimax con Poda Alfa-Beta

Para dotar a la máquina de capacidad de decisión, se ha implementado un agente basado en el algoritmo Minimax complementado con la optimización de poda alfa-beta. Dado que el espacio de búsqueda de Oteló es considerablemente grande, el algoritmo se ejecuta con una profundidad limitada. Al alcanzar dicha profundidad, se aplica una función heurística de evaluación manual básica, consistente en la diferencia de fichas entre el jugador maximizador y el minimizador.

Durante la implementación del agente, se identificaron y resolvieron dos desafíos arquitectónicos principales para asegurar la robustez del agente:

- **Gestión de turnos nulos en simulación:** En partidas avanzadas, es posible alcanzar estados donde un jugador no tiene movimientos válidos. Se implementó la lógica recursiva necesaria para que el agente ceda el turno de forma virtual en su árbol de búsqueda, en lugar de evaluar erróneamente esa rama como una derrota inmediata. Esto evita colapsos lógicos y la devolución de jugadas nulas al motor gráfico.
- **Ruptura del determinismo:** Al depender de una heurística simple de conteo de fichas, el agente podía generar movimientos repetitivos en situaciones de empate heurístico. Se introdujo un mecanismo de aleatorización en la selección de movimientos cuando múltiples opciones presentaban la misma evaluación. Esta imprevisibilidad es un requisito técnico fundamental para garantizar una generación de datos heterogénea y rica durante la fase de autójuego para el posterior entrenamiento de la red neuronal.

III-D. Generación del Conjunto de Datos mediante Autojuego

Para entrenar la red neuronal, es necesario disponer de un conjunto de datos etiquetado que relacione estados del tablero con la calidad de la posición para el jugador activo. Dado que no existen bases de datos públicas de partidas de Oteló, se ha desarrollado un módulo de autojuego que enfrenta al agente Minimax contra sí mismo.

Se diseñó un entorno de simulación aislado de la interfaz gráfica donde dos agentes Minimax compiten entre sí. Durante la ejecución de cada partida, se registran los estados del tablero antes de cada movimiento junto al jugador activo.

Una vez finalizada la partida, se determina el ganador y se recorre el historial de estados para etiquetar cada posición con el resultado final desde la perspectiva del jugador activo en ese momento:

- **+1:** Si el jugador activo ganó la partida.
- **0:** Si la partida terminó en empate.
- **-1:** Si el jugador activo perdió la partida.

Aprovechando la aleatoriedad introducida previamente en el agente para romper el determinismo, se simuló 500 partidas completas de forma automatizada, generando una base de datos heterogénea de aproximadamente 30.000 tableros. Para optimizar el rendimiento de lectura durante la fase de entrenamiento, las estructuras de datos se transformaron en tensores y

se exportaron en formato binario nativo (.npz) haciendo uso de la librería NumPy, separando las características de entrada (matrices de estado) de las etiquetas de salida.

III-E. Arquitectura y Entrenamiento de la Red Neuronal

Una vez generado el conjunto de datos, se procedió a diseñar, compilar y entrenar una red neuronal profunda para aprender a evaluar las posiciones del tablero. Para ello, se utilizó la interfaz de Keras sobre el motor TensorFlow. Siguiendo un enfoque alineado con las prácticas de la asignatura, se implementó un modelo secuencial con la siguiente arquitectura:

- **Capa de Entrada y Aplanado:** Recibe la matriz del tablero de 8×8 casillas, la cual es procesada mediante una capa de aplanado (Flatten) para convertirla en un vector unidimensional de 64 elementos.
- **Capas Ocultas:** Dos capas densas (Dense) con 64 y 32 neuronas respectivamente, ambas con función de activación ReLU para introducir no linealidad y evitar la saturación de gradientes.
- **Capa de Salida:** Una única neurona con función de activación de tangente hiperbólica (tanh), que devuelve un valor continuo en el rango $[-1, 1]$, representando la evaluación de la posición desde la perspectiva del jugador activo.

Para el proceso de optimización, se seleccionó el algoritmo de Descenso Estocástico por el Gradiente (SGD) con un factor de aprendizaje fijo de 0.01. La función de pérdida elegida fue el Error Cuadrático Medio (MSE), evaluada conjuntamente con la métrica del Error Absoluto Medio (MAE). El entrenamiento se llevó a cabo durante 50 épocas con un tamaño de lote de 256 ejemplos, aplicando una división de 80 % para entrenamiento y 20 % para validación.

IV. PRUEBAS Y EXPERIMENTACIÓN

IV-A. Estudio de Hiperparámetros y Selección del Modelo

Con el objetivo de optimizar la capacidad de generalización del modelo, en la fase de pruebas no se entrenó un único modelo, sino que se realizó un estudio comparativo evaluando tres configuraciones distintas de hiperparámetros.

Para garantizar la reproducibilidad del experimento, se fijaron las semillas de pseudoaleatoriedad ($seed = 42$) en las librerías NumPy y TensorFlow, asegurando que el reparto del 20 % de los datos de prueba fuera idéntico para todos los candidatos. Los tres modelos evaluados, todos con una estructura de 64 y 32 neuronas en sus capas ocultas, fueron los siguientes:

- **Modelo A:** Arquitectura base con función de activación ReLU y optimizador SGD con tasa de aprendizaje de 0.01.
- **Modelo B:** Mismo optimizador, pero sustituyendo ReLU por la función de activación Sigmoide en las capas ocultas.
- **Modelo c:** Arquitectura base (ReLU), pero sustituyendo el optimizador clásico por el algoritmo Adam con tasa de aprendizaje de 0.001, buscando una convergencia más rápida y estable.

Tras someter a los tres modelos a 50 épocas de entrenamiento, los resultados del error absoluto medio (MAE) sobre el conjunto de validación fueron los siguientes:

- Modelo A (Base): MAE de 0.9848
- Modelo B (Sigmoide): MAE de 0.9797
- Modelo C (Adam): MAE de 1.0384

El **Modelo C** experimentó un fenómeno de sobreajuste, con un error de validación superior al de entrenamiento, lo que indica que el optimizador Adam no fue capaz de generalizar adecuadamente en este caso. Por otro lado, el **Modelo B** mostró una ligera mejora respecto al modelo base. No obstante, el error cercano a 1.0 sugiere que la red no está aprendiendo patrones significativos y, ante la duda, tiende a predecir la situación del tablero como un empate (0). Esta limitación está relacionada con el uso de la capa Flatten, que no preserva la estructura espacial del tablero, dificultando que la red capture relaciones locales entre las fichas.

Como prueba final para descartar que el error del modelo se debiera a una falta de capacidad paramétrica o a un entrenamiento demasiado corto, se diseñó un experimento límite (Modelo D). Se construyó una topología masiva, incluyendo capas de BatchNormalization y Dropout, y se extendió el entrenamiento hasta las 500 épocas. Los resultados confirmaron el diagnóstico previo: la red alcanzó un error de entrenamiento bastante mejor que los previos (MAE de 0.7739), pero falló drásticamente en el conjunto de validación, disparando su error hasta 1.0058. Este comportamiento evidencia otro caso de sobreajuste. Al carecer de visión geométrica por el aplanado de los datos, y al tener tanta capacidad de memoria, la red optó por memorizar los tableros exactos del entrenamiento, perdiendo toda capacidad de generalizar reglas estratégicas.

A pesar de esta limitación, el **Modelo B** demostró ser la aproximación más estable y fue seleccionado como el modelo definitivo (otelo_B.keras) para competir en el torneo final.

V. CONCLUSIONES

VI. BIBLIOGRAFÍA

REFERENCIAS

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020, ch. 5, "Adversarial Search".

REFERENCES

Please number citations consecutively within brackets [?]. The sentence punctuation follows the bracket [?]. Refer simply to the reference number, as in [?]; do not use "Ref. [?]" or "reference [?]" except at the beginning of a sentence: "Reference [?] was the first ..."

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors' names; do not use "et al.". Papers that have not been published, even if they have been submitted for publication, should be cited as "unpublished" [?]. Papers that have been accepted for

publication should be cited as “in press” [?]. Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation [?].

REFERENCIAS

- [1] World Othello Federation, “Othello Rules,” [Online]. Disponible en: <https://www.worldothello.org/about/about-othello/othello-rules>

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template text from your paper may result in your paper not being published.